

UNIVERSITÀ DI PISA

Decentralised Lending Service

Marco Pernisco / Student ID 683674 P2P Systems and Blockchains Lab 2025/26

Source code: `https://github.com/Marcop-00/Decentralised-Lending-Service`

1. Architecture and protocol

The service pools contributors' ETH and lets them vote on loans. A **contributor** deposits, votes, withdraws disposable capital and claims interest or compensation. An **applicant** proposes terms, resolves the proposal and, if funded, receives and repays ETH. One account may hold both roles.

On-chain design. `FundingPoolProxy` holds ETH and state; `FundingPool` supplies UUPS logic. Only the owner may upgrade it; tests check state preservation. Each approved loan deploys a separate `LoanContract` with its own borrower, terms and repayment state. Existing loan bytecode does not change when the pool is upgraded. `BitcoinOracle` stores satoshi balances and emits paid update requests. The pool reads its stored balance during proposal resolution.

Off-chain design. The trusted Python oracle processes the first **131,000 Bitcoin mainnet blocks**, one block at a time, rebuilding the UTXO index **in memory**. It then answers Ethereum update requests. Restarting repeats the scan: no Bitcoin balance/UTXO cache is stored on disk. The separate voter detects proposal events and always votes YES, even with fully locked capital. SQLite persists service events and signed transactions, not Bitcoin balances; retries survive temporary RPC failures.

Trust model. BTC ownership is not proved and BTC are not locked. Historical Bitcoin liquidity is evaluated at the fixed rate **1 BTC = 30 ETH**. The parser checks connected blocks, hashes and Merkle roots, excludes the unspendable genesis reward and refuses incomplete input; it is not a full Bitcoin consensus node.

Voting and funding

Deposits must exceed **100,000 wei**. Voting eligibility includes locked capital, but resolution weights each contributor's **current disposable balance**; nonvoters count as NO and ties reject. Only the original applicant may resolve, after at least **12 blocks**. Voting remains open until resolution; `votingDeadlineBlock` names the earliest resolution block. All durations are block-height differences. Applicants specify principal, interest rate (1–100), duration and a BTC address.

Insufficient disposable ETH or Bitcoin liquidity rejects and closes the proposal. Approval closes voting and locks capital proportionally across contributors. Integer-division remainders reduce the **actual disbursed principal**; no arbitrary participant receives the difference. The loan fixes its collateral percentage at creation.

Repayment, interest and compensation

Capital is either **disposable** or **locked**; claimable interest and the compensation reserve are separate, non-loanable balances. Only the original borrower may repay, including after default. Each installment covers principal and interest in proportion to the total debt; one payment may repay everything.

Principal refunds unlock contributor capital in the pool, not directly in their wallets. Refunds follow descending *initial* loan shares, with ties broken by descending numerical address. The order is sorted once. **Interest** is split using the loan's fixed collateral percentage: collateral enters the reserve; the rest becomes contributor gains proportional to initial shares, collected through `claimInterestGain()`. Cumulative payment calculations preserve principal, interest and collateral rounding across installments. Surplus above the full debt and interest distribution dust enter the reserve. Page 4 gives a numerical example.

Success/default. Full repayment by the due block succeeds; an overdue, unpaid loan is marked failed by a transaction, not by time passing alone. Failure is counted once and cannot be reversed by late repayment. Global collateral starts at **50%**, stays within **1–100%**, decreases five points per success and increases five per failure; existing loans keep their original percentages.

Compensation covers unpaid principal only, capped by the reserve. An empty-reserve request still records default, pays zero and preserves the claim for future refills. Actual compensation forfeits the same amount of future principal recovery without changing original interest weights. Only the owner may **terminate** the service, with no active loans; withdrawal, interest claims and recovery remain available.

2. Security and operational limits

Reentrancy: modification and executable attack

The test-only `FundingPoolVulnerable` inherits the real pool and replaces withdrawal with stale balance writes **after** the external transfer:

```
uint256 beforePosition = totalContributed[msg.sender];
uint256 beforePool = totalUnlockedFunds;
(bool sent, ) = payable(msg.sender).call{value: amount}("");
if (!sent) revert TransferFailed();
totalContributed[msg.sender] = beforePosition - amount;
totalUnlockedFunds = beforePool - amount;
```

`ReentrancyAttacker` recursively calls `withdraw(amount)` from its receive function. In `test/reentrancy.test.ts`, a 1 ETH attacker deposit drains the honest contributor's 10 ETH from the vulnerable proxy; the same attacker cannot steal those 10 ETH from the normal proxy, whose accounting changes before transfer. Simply subtracting from a live balance after transfer could revert on underflow in Solidity 0.8; stale assignments make the modified example actually exploitable. Test variants are never deployed by service setup. These checks are not a general security proof.

Unfair remuneration without reentrancy

A majority contributor with **6 ETH** can approve a **10 ETH** loan to a colluding borrower, backed also by an honest contributor's **4 ETH**, then default. The colluding group captures a net **4 ETH**, not 10 ETH, before gas and other flows. Any sufficiently funded BTC address can be cited because ownership is not proved. Priority refunds also favor large positions: with backing 6/4, a base repayment of 6 restores the larger position before the smaller contributor recovers principal. These follow the protocol's economic rules, rather than a reentrancy flaw.

Limits. Empty positions are removed, but work scales with contributors and must fit block gas limits. Checkpoints detect chain replacement; nonce conflicts remain inspectable. Reorganization recovery is not automatic.

3. Run the service

Run commands from the **project root**, unless indicated otherwise. Use a supported even-numbered Node release (22.13.0+), Python 3.10+ and **Geth 1.13.14**, the pre-PoS Clique client matching the private chain's genesis.

1. Install dependencies and validate Bitcoin data

```
./setup.sh
./.venv/bin/python python/oracle/check_data.py --data-dir bitcoin --summary
```

Setup installs locked Node dependencies and Python packages, creates `.venv`, compiles contracts and obtains Geth. Installation and uncached compiler downloads need internet access; skip setup if already completed. Supply decoded mainnet `blk*.dat` files containing genesis through height 130,999. The offline checker needs no keys or Geth: success reports `valid: true`, **131,000 blocks** and height **130,999**, and lists funded addresses. Missing/incomplete data must not be replaced by fake balances.

2. Start the private chain / terminal 1

```
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore --networkid=202526 \
  --http --http.addr=127.0.0.1 --http.port=8545 --http.api=eth,net,web3 \
  --mine --miner.gasprice=100000000 \
  --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

Keep the node running.

3. Create accounts and deploy / terminal 2, project root

```
./.venv/bin/python python/setup.py
```

The provided prefunded account **only transfers ETH** to three newly generated accounts. A new deployer creates the oracle, implementation and initialized proxy. Setup rebuilds ordinary bytecode, checks chain ID, receipts, code and initialization, then writes protected service environments including `DEPLOYMENT_BLOCK`. Repeating setup creates a separate deployment.

4. Start the oracle and voter / separate terminals

```
./.venv/bin/python python/oracle/oracle_service.py
./.venv/bin/python python/voter/auto_voter.py
```

Wait for **ORACLE READY**: the oracle must finish its initial in-memory Bitcoin scan. There is no fixed sleep to configure; restarting repeats the scan. Keep the services running during the demo. Preserve each service's SQLite event queue after temporary RPC failures; it is separate from Bitcoin indexing.

5. Run repayment and default / another terminal, project root

```
./.venv/bin/python python/simulate.py \
  --btc-address 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa --principal 1
```

Parameter. `-principal` sets the first loan's requested principal in **ETH**, excluding interest (default: 1 ETH). It is not a BTC amount or the contributor's deposit. Scenario B uses half that principal, rounded down to wei with a minimum of 1 wei.

This concrete address has **4,000,000 satoshi** in the validated snapshot, enough for 1 ETH at the fixed rate. Scenario A repays a 1 ETH loan at 5% in two installments and claims gain. Scenario B leaves 0.5 ETH unpaid, requests compensation and retries with an empty reserve. Both proposals wait for YES votes and 12 real blocks. Balances, loan states and accounting checks are printed after significant changes.

Timing and recovery. Clique blocks take about 10 seconds: allow several minutes for both voting periods. `-timeout` sets the maximum seconds *per asynchronous step* (default 300). On timeout, check node and service logs before retrying; do not delete chain data, keys, environments or event queues. The expected accounting is explained on page 4; reused deployments may differ.

4. Demonstration: repayment and default

This observed local run uses `-principal 1`. The fresh oracle response is **4,000,000 satoshi = 0.04 BTC**, equivalent to **1.2 ETH** of capacity at 30 ETH/BTC. BTC are not transferred or locked. The contributor deposits **50 ETH**; its wallet falls from 100 ETH to roughly 50 ETH after gas, while the pool holds 50 ETH of disposable capital.

First loan: 1 ETH, 5% interest, two installments

After the YES vote and 12-block period, approval creates and funds a dedicated loan. The pool holds 49 ETH; the contributor has **49 disposable + 1 locked**. The debt is **1.05 ETH**. With the loan's **50% collateral**, each 0.525 ETH installment restores 0.5 ETH of principal and pays 0.025 ETH of interest, divided equally between reserve and contributor gain:

Destination	Each installment (ETH)	Both (ETH)
Principal restored to disposable capital	0.5000	1.0000
Interest credited to compensation reserve	0.0125	0.0250
Interest credited as contributor gain	0.0125	0.0250
Borrower payment	0.5250	1.0500

The loan becomes `Repaid`: disposable capital returns to 50 ETH and locked capital to zero. The contributor claims the 0.025 ETH gain; the pool retains **50 ETH capital + 0.025 ETH reserve**. Global collateral decreases from 50% to **45%**, while the first loan's own percentage remains 50%.

Second loan: expiry does not execute a transaction

A second proposal receives YES and is resolved after another 12-block period. It disburses **0.5 ETH** at 10% interest, with collateral fixed at **45%**, and is deliberately not repaid. It is funded at block **328** and due at **331**. At block 332 it is overdue but still stored as `Funded`: merely passing a block height does not execute code. The compensation transaction at block **333** records `Defaulted`, reduces active loans to zero, counts one failure and raises global collateral back to **50%**. The second loan's own percentage stays 45%.

Compensation: limited reserve and preserved claim

Unpaid principal is 0.5 ETH, but the reserve holds only 0.025 ETH. The contributor receives **0.025 ETH directly in its wallet**; the remaining principal claim is **0.475 ETH**. The retry at block **334** pays zero without erasing this claim or counting the default twice.

Pool accounting (ETH)	Before compensation	After / retry
Contributor total position	50.000	49.975
Contributor disposable capital	49.500	49.500
Contributor locked principal	0.500	0.475
Compensation reserve	0.025	0
Actual ETH held by the pool	49.525	49.500

Interpretation. The 0.475 ETH locked position is an **unrecovered claim, not withdrawable cash**. Compensation reduces both total position and locked principal, leaving disposable capital unchanged: $49.975 - 0.475 = 49.5$ ETH. Future reserve refills or late repayments may allow further recovery, but it is not guaranteed. **Zero active loans does not mean zero loss.**

Wallet balances and transaction count

The contributor's final wallet is **50.0499454037 ETH**, including gas: interest gain and compensation arrived there, while 49.5 ETH remain disposable in the pool. This run adds **11 demo transactions + 1 oracle update + 2 YES votes = 14**; setup adds six (three transfers and three deployments). The oracle holds 0.0000098852 ETH because an earlier unsuccessful address request also paid its fee. That earlier request/response pair is outside this run's 14 transactions. Block numbers and wallet values belong to this observed run; reused deployments and different input can change them.

5. Testing and gas evaluation

5.1 Test execution and results

After dependency setup (page 3), run from the **project root**. **No Geth, oracle, voter, keys, service environments or real Bitcoin files are needed.** Hardhat uses isolated simulated chains; Python tests use temporary data and mocked external services. They do not alter the existing private-chain deployment. Do not run `python/setup.py` just to test.

Compilation and complete test suites

```
npm run typecheck
npm test
./.venv/bin/python -m unittest discover -s python/tests -v
```

Typecheck: builds contracts and runs `tsc -noEmit`; successful exit with no errors is sufficient, even if TypeScript prints nothing. **Contracts:** expect **34 passing**, comprising **4 Solidity + 30 Node.js/TypeScript tests**. A Solidity fuzz test uses **256 generated inputs**; these are repetitions of one test, not 256 extra tests. **Python:** expect **Ran 49 tests** and **OK**.

Contract scenarios cover voting boundaries and changing balances, BTC thresholds, allocation dust, installments, refund order, interest, defaults, repeated compensation, access, termination, upgrades and reentrancy. Explicit checks validate event emitters/arguments and independent contracts for two borrowers. Python checks cover parsing, genesis exclusion, UTXOs, atomic indexing, durable event/transaction retries, the always-YES voter, setup safeguards and demo compensation. Mocked balances test contract logic, not the real Bitcoin dataset.

Select a focused check

```
npm test -- test/advanced_behaviors.test.ts
npm test -- test/events_and_isolation.test.ts
npm test -- test/reentrancy.test.ts
npm test -- test/upgrade.test.ts
npm test -- contracts/test/LoanContract.t.sol
./.venv/bin/python -m unittest discover -s python/tests \
  -p test_demo_scenarios.py -v
```

Choose the relevant command: accounting; events and loan isolation; reentrancy; upgrades; native loan rules; or **six offline demo checks**. Event checks include a compensation retry that must not emit another default. Isolation checks repay two loans with different borrowers and ensure that neither changes the other loan's terms, repayments or contributor positions.

Inspect code coverage

```
npm run test:coverage
```

Coverage: expect **34 passing** and line/statement percentages. Open `coverage/html/index.html` in a browser. This is contract coverage, not Python coverage or proof of security. Instrumented bytecode changes gas: coverage deliberately leaves ordinary gas measurements unchanged. The profiling commands and measurements are on page 6; do not use coverage gas for costs.

Recognize success and troubleshoot

A test may deliberately trigger a revert and still pass: it checks rejection of an invalid operation. The reentrancy test passes only if the unsafe variant is drained *and* the normal pool protects honest funds. Read the final summary; failed assertions, `FAIL`/`ERROR` or a nonzero exit status need investigation. Counts describe this version and may change as tests are added.

Missing modules/interpreter: check the working directory, supported Node version and completed `./setup.sh`; use `./.venv/bin/python`. **Failed test:** rerun its file and inspect the first expected/actual mismatch. Do not delete private-chain data or event queues to fix isolated tests. **Missing Bitcoin input:** repair the dataset for the separate checker/demo (page 3); passing mocked tests does not validate those files. The real demo is an integration exercise, not a substitute for automated tests or vice versa.

5. Testing and gas evaluation (continued)

5.2 Gas measurement and interpretation

Functional tests check that operations behave correctly; gas profiling measures their execution cost. Both use the same Hardhat project and contract implementation, with ordinary bytecode for measurements.

Environment. Hardhat 3.18.1, TypeScript 6.0.3, `node:test`, viem 2.57.3; Solidity 0.8.24, optimizer 200 runs, EVM/hardfork **Berlin**, simulated chain ID 202526. The scenario has one contributor.

Reproduce the measurements

```
npm run test:gas
```

Expect **1 passing** profiling test covering **20 operation scenarios**. It records receipt `gasUsed`, including intrinsic gas, in `docs/gas-measurements.json`, created automatically when the test runs. The table below presents the measured results for this implementation. The full `npm test` run also includes profiling and rewrites the JSON.

Measured gas costs

Operation	Scenario	Gas units
FundingPool / via proxy		
`contribute()`	New contributor	140,923
`withdraw()`	Partial withdrawal	47,933
`withdraw()`	Final withdrawal	32,412
`createLoanProposal()`	New proposal	234,798
`voteOnProposal()`	YES vote	106,936
`executeApprovedLoan()`	Approved	1,049,274
`executeApprovedLoan()`	Rejected	90,860
`claimInterestGain()`	Claim accrued gain	24,691
`requestCompensation()`	Reserve pays claim	52,652
`requestCompensation()`	Reserve empty	43,786
`upgradeToAndCall()`	Compatible V2	37,769
`terminate()`	No active loans	36,295
LoanContract		
`repay()`	Partial installment	168,082
`repay()`	Final installment	90,716
`repay()`	Late repayment	138,782
`markDefaulted()`	Overdue loan	103,758
BitcoinOracle		
`requestUpdate()`	Paid request	25,193
`update()`	First nonzero balance	49,426
`update()`	Existing balance	32,326
`withdrawFees()`	Operator collects fees	30,868

Read the values and calibrate the oracle fee

Values are **gas units**, not wei and not the submitted transaction gas limit. The fee in wei is **gas used times gas price**. Getters invoked through `eth_call` incur no paid transaction fee.

Oracle fee: $49,426 \times 0.1$ gwei $= 4,942,600,000,000$ wei. Calibration uses a first nonzero update of a 34-character historical address; existing-slot updates cost less. Bytecode/settings changes require recalibration.

Scope. These are reproducible scenario measurements, not universal upper bounds. Costs depend on storage state and contributor count; in particular, pool work can grow with the number of contributors. Coverage uses instrumented bytecode, so it exercises these paths without overwriting the ordinary gas JSON. Use the profiling command above for cost evaluation.